

**DEPARTMENT OF COMPUTER SCIENCE & INFORMATION
TECHNOLOGY**

OBJECT ORIENTED PROGRAMMING

Course Code: CT-260

**ASSIGNMENT — DESIGN PATTERNS IN REAL-
WORLD SOFTWARE SYSTEMS**



First Year | Section B

Submitted By:

Arbish Tehseen (CT-25053)

Areefa Samar (CT-25062)

Saita Ahuja (CT-25065)

Task A: Application Overview

1. What Does the Application Do?

Duolingo is a free language learning platform that teaches users new languages through short, interactive, game-like lessons. The app gradually builds vocabulary, grammar, pronunciation, and conversational fluency helping complete beginners progress to confident language use at their own pace.

The learning is structured into Courses, Sections, Units, and Lessons, each building on the previous. Inside lessons, users complete exercises such as translation, multiple choice, speaking, matching, and fill in the blank, targeting reading, writing, listening, and speaking skills. Users earn XP points, maintain daily streaks, and unlock new content as they progress.

Beyond individual learning, Duolingo includes social and competitive features where users follow friends, compete in weekly leagues, and climb leaderboards. The app also offers a premium subscription called Duolingo Super, which removes advertisements, enables offline learning, and provides unlimited practice.

Duolingo supports over 40 languages including Spanish, French, Japanese, Arabic, and Mandarin and has over 500 million registered users worldwide, making it one of the most widely used language learning platforms in the world.

2. Who Are Its Users?

Duolingo's user base is very broad and diverse. The main users are casual learners, which includes students, working professionals, and hobbyists who want to learn a new language at their own pace without attending formal classes. Many of its users are school and university students who use it alongside their studies to practice a language they are already learning in class. There are also adult users who are learning a language for a specific reason such as travel, job opportunities, moving to a new country, or simply as a personal hobby.

Since the app has a very simple and colorful design with a game-like feel, it is also very popular among younger audiences and even children. The app's structure makes it easy enough for beginners of any age to start without any prior knowledge of the language.

Beyond individual users, Duolingo also has a version called Duolingo for Schools, which is used by teachers to assign lessons to their students and track their progress, making its user base extend into classrooms as well.

Overall, Duolingo is designed to be accessible to everyone regardless of age, background, or education level. With over 500 million registered users worldwide, it serves one of the largest and most diverse audiences of any educational app in the world.

3. Core Features and Modules

Duolingo is composed of eight tightly integrated modules. Each module handles a distinct concern and maps to one or more OOP design principles.

Module	Description & OOP Hook
User Authentication & Profile	Account creation, login/logout, OAuth (Google/Apple), profile management, language preferences, and daily goal settings. Encapsulation all user data is private and exposed only through controlled getters/setters.
Course & Curriculum Catalogue	Hierarchical tree of 40+ languages: Course > Section > Unit > Lesson > Exercise. Each level is selectable and progressively unlocked. Composite Pattern every node implements the same LearningComponent interface.
Exercise Engine	Generates and evaluates 8+ exercise types: translation, fill-in-the-blank, listening, speaking, matching, and word-bank drag-and-drop. Factory + Strategy Patterns ExerciseFactory creates typed exercises; EvaluationStrategy selects the correct grading algorithm.
Gamification & Rewards	XP, streak tracking, leaderboards, badges, achievement milestones, lingots, and animated level-up celebrations. Observer Pattern: StreakObserver, XPObserver, BadgeObserver all subscribe to lesson-complete events.
Notification System	Push notifications for streaks, reminders, friend challenges, and league updates. Observer + Strategy: notification delivery strategy is swappable at runtime.
Answer Validation Pipeline	Multi-layer validation: empty check, length check, character-set check, grammar check, and semantic NLP scoring. Chain of Responsibility each validator is an independent handler.
Session & Progress Manager	Saves lesson checkpoints, restores sessions after app exit, supports mid-lesson retry, and syncs progress across devices. Memento Pattern: LessonMemento snapshots encapsulate full session state.
Cross-Platform UI Layer	Adaptive platform-specific UI for Android, iOS, and Web, all sharing a common interface contract. Abstract Factory Pattern: AndroidUIFactory, iOSUIFactory, and WebUIFactory each produce a consistent family of UI components.

4. Why Did We Choose Duolingo?

Choosing the right application for an OOP patterns assignment is itself a design decision. We evaluated several candidates: a banking system, a social media platform, a hospital management system and chose Duolingo for the following concrete, technical reasons:

#	Reason
1	Rich OOP Architecture, Duolingo is a full-stack, production-grade system. Its layers: UI, business logic, data, gamification map almost perfectly onto core OOP design principles, making every pattern feel natural rather than forced.
2	8 GoF Patterns, All Visible, The application natively exhibits Factory, Observer, Flyweight, Composite, Strategy, Abstract Factory, Memento, and Chain of Responsibility , one pattern per major subsystem. No artificial examples needed.
3	Real-World Relevance, over 500 million registered users interact with these exact systems daily. The patterns studied here are not academic exercises, they power live software at massive scale.
4	Encapsulation Done Right, Duolingo's sealed Memento, private session data, and interface-only access between layers are textbook demonstrations of encapsulation and information hiding.
5	Extensibility as a Feature. Adding a new language, exercise type, validator, or notification channel requires only a new class, never a modification of existing ones. This is the Open/Closed Principle made visible.
6	Relatable Problem Domain, every student has used a language-learning app. The domain is immediately understandable, so focus can stay fully on the patterns themselves rather than explaining the application context.

5. Design Pattern to Module Mapping

The table below maps all eight GoF patterns covered in this assignment to their respective Duolingo modules.

Pattern	Duolingo Module	Category
Chain of Responsibility	Answer Validation Pipeline	Behavioral
Composite	Course Catalogue Hierarchy	Structural
Observer	Gamification & Rewards	Behavioral
Flyweight	Letter Tile Renderer	Structural
Memento	Session & Progress Manager	Behavioral
Abstract Factory	Cross-Platform UI Layer	Creational
Strategy	Exercise Evaluation	Behavioral
Factory Method	Exercise Engine	Creational

Task B : Design Pattern Identification

Pattern 1 — Chain of Responsibility (Behavioral)

Pattern Name: Chain of Responsibility: decouples the sender of a request from its receivers by giving multiple objects the chance to handle the request one by one, passing it along a chain until one of them either handles or rejects it. Each handler decides independently: process this, or pass it to the next handler.

Category: Behavioral: these patterns define how objects collaborate and share responsibilities. Chain of Responsibility specifically addresses routing a request through a dynamic sequence of independently encapsulated processing steps.

Where and How Does This Pattern Appear in Duolingo?

This pattern appears in two subsystems within Duolingo: the Answer Validation Pipeline and the User Support Escalation System.

Answer Validation Pipeline: When a user submits an answer, it passes through a carefully ordered chain of AnswerValidator handlers. EmptyAnswerValidator checks if the answer is blank. LengthValidator rejects oversized inputs. CharacterSetValidator ensures only valid language characters are present. SyntaxValidator checks basic grammatical structure. Finally, SemanticAnswerValidator uses NLP-based comparison to determine semantic equivalence with the expected answer. Each validator is assembled by the ExerciseController at exercise load time using setNext().

User Support Escalation System: Support requests travel through a similar chain: AutomatedFAQHandler attempts keyword-matched resolution first, followed by ChatbotHandler, then CommunityModeratorHandler, and finally HumanSupportAgentHandler for live resolution.

Classes and Modules Involved

- **AnswerValidator (abstract handler):** base abstract class defining setNext(handler) to link handlers together and validate(answer) as the abstract processing method.
- **EmptyAnswerValidator, LengthValidator, CharacterSetValidator, SyntaxValidator, SemanticAnswerValidator:** concrete handlers. Each implements validate(answer). If validation passes, delegates to next; if it fails, returns an error immediately.
- **ExerciseController (chain builder/client):** constructs the validation chain at exercise load time by linking handlers in the correct order using setNext().

Why Chain of Responsibility? What Problem Does It Solve?

Without this pattern, the ExerciseController would contain deeply nested if-else conditions handling every validation rule in one place making it enormous, fragile, and nearly impossible to maintain. Adding a new rule such as a ProfanityFilterValidator would require modifying the ExerciseController directly, violating the Open/Closed Principle. The Chain of Responsibility distributes validation across independently maintained, single-purpose handler classes. The ordering is also performance-optimised: the cheapest checks run first and fail early, ensuring the expensive SemanticAnswerValidator only runs when truly necessary.

OOP Principles Reinforced

- **Abstraction:** The ExerciseController interacts only with the abstract AnswerValidator interface. It has no knowledge of how many handlers exist or what each one does internally.
- **Single Responsibility Principle:** Each handler class has exactly one validation job. No handler crosses into another's responsibility, making each independently testable.
- **Open/Closed Principle:** The system is open for extension, new validators are added as new classes. Existing handlers and the ExerciseController require zero changes.
- **Polymorphism:** The ExerciseController calls validate() on the first handler through the abstract interface. Each concrete handler overrides this method with its own logic, resolved at runtime.

Pattern 2 — Composite (Structural)

Pattern Name: Composite: builds tree-like structures of objects and treats every single node whether a single tiny object or a giant group containing hundreds of objects in the exact same way through a common interface. The whole point is uniformity.

Category: Structural: these patterns are about how classes and objects are organized to form larger structures, keeping the system flexible. Composite specifically organizes objects into trees to represent nested structures.

Where and How Does This Pattern Appear in Duolingo?

Duolingo's entire learning content is organized as a deep hierarchical tree. Every node in this tree from one tiny Exercise at the bottom to the entire Course at the top implements the same interface: LearningComponent.

```
Course  ("Spanish for English Speakers")
|
|-- Section  ("Basics")
|   |
|   |-- Unit  ("Unit 1: Greetings")
|   |   |
|   |   |-- Lesson  ("Lesson 1")
|   |   |   |
|   |   |   |-- Exercise  (Translate "Hello")
|   |   |   |-- Exercise  (Multiple choice)
|   |   |   |-- Exercise  (Speaking)
|   |   |
|   |   |-- Lesson  ("Lesson 2")
|   |   |   |
|   |   |   |-- Exercise  (Match the pairs)
|   |   |   |-- Exercise  (Fill in the blank)
|   |
|   |-- Section  ("Travel")
```

The LearningComponent interface declares: start(), isCompleted(), getProgress(), getTotalXP(), getTitle(). There are three types of nodes in this tree:

- **LearningComponent (Abstract Class):** the base class every node inherits from. Declares all common methods. Because every class inherits from this, the entire system can treat an Exercise and a Course identically.
- **Leaf Node — Exercise:** the most basic unit, cannot contain anything inside it. `isCompleted()` checks if this one question was answered. `getTotalXP()` returns its own fixed XP value.
- **Composite Nodes — Lesson, Unit, Section, Course:** contain children and handle every method by delegating down. `Lesson.isCompleted()` checks if all its Exercises are done. `Course.getTotalXP()` recursively adds up XP from every node all the way down.

Classes and Modules Involved

- **LearningComponent:** abstract base class declaring `start()`, `isCompleted()`, `getProgress()`, `getTotalXP()`, `getTitle()`.
- **Exercise:** leaf class. Represents one single question or task. Has no children.
- **Lesson:** composite class. Contains a list of Exercise objects. Iterates over them to implement all methods.
- **Unit:** composite class. Contains a list of Lesson objects. Delegates all method calls down.
- **Section:** composite class. Contains a list of Unit objects. Delegates all method calls down.
- **Course:** root composite class. Contains a list of Section objects. Top of the entire tree.
- **ProgressTracker, XPRewardSystem, UIRenderer:** client modules that hold a LearningComponent reference and call methods on it uniformly.

Why Composite? What Problem Does It Solve?

Without Composite, every module: ProgressTracker, XPRewardSystem, UIRenderer would need completely different code for every level of the hierarchy:

```
if (component is Exercise) { check exercise directly }
else if (component is Lesson) { loop through exercises }
else if (component is Unit) { loop through lessons, then exercises }
else if (component is Course) { loop through sections, units, lessons, exercises }
```

Every operation, checking completion, calculating XP, rendering a progress bar would need its own version of this. Adding a new level like 'Skill Block' between Unit and Lesson means updating every single module everywhere. With Composite, the entire system uses one uniform call `learningComponent.start()` and the value bubbles back up automatically. Zero if-else chains. Zero nested loops.

OOP Principles Reinforced

- **Abstraction:** Clients like ProgressTracker and UIRenderer never know whether they are talking to an Exercise or a Course just the abstract interface.

- **Polymorphism:** When ProgressTracker calls component.getProgress(), the correct version executes automatically. Same call, different behavior, zero if-else.
- **Encapsulation:** Each node manages its own data privately. Lesson internally manages its exercises — no outside code touches that list.
- **Open/Closed Principle:** Adding a new level like SkillBlock only requires one new class inheriting from LearningComponent. No existing code changes.

Pattern 3 — Observer (Behavioral)

Pattern Name: Observer: sets up a one-to-many relationship. When one main object (the Subject) changes, it automatically notifies a whole list of other objects (the Observers).

Category: Behavioral: these patterns focus on how objects communicate and share responsibility. They help manage complex communication so that objects do not have to be tightly coupled just to share data.

Where and How Does This Pattern Appear in Duolingo?

This pattern is the foundation of Duolingo's Gamification and Progress System. When a user completes a lesson, multiple different parts of the app need to react, the streak needs to update, XP needs to be added, badges need to be checked, a push notification needs to be sent, and analytics need to log the event.

The LessonController acts as the Subject and maintains a list of registered Observers. When a lesson is completed, it calls notifyObservers() with a LessonCompletedEvent object. Every registered observer receives this and handles it independently. Each observer implements a common Observer interface with an update(event) method: StreakObserver updates the streak, XPObserver adds points, BadgeObserver checks for achievements, and so on.

Classes and Modules Involved

- **LessonController (Subject/Publisher):** fires events on lesson completion.
- **Observer (interface):** defines the update(event) method contract.
- **StreakObserver, XPObserver, BadgeObserver, NotificationObserver, AnalyticsObserver:** concrete observers, each reacting independently.
- **LessonCompletedEvent:** the event payload object carrying relevant data (user ID, XP earned, lesson ID, timestamp).

Why Observer? What Problem Does It Solve?

If the Observer pattern were not used, the LessonController would have to call each of these modules directly, creating tight coupling between them. Adding any new feature would require modifying the LessonController every single time. With Observer, the lesson code does not need to know how the leaderboard or streaks work, it just sends a signal and the other parts of the app listen and react independently.

OOP Principles Reinforced

- **Abstraction:** The subject only knows about the Observer interface, never about concrete observer classes, making the system modular and easy to extend.
 - **Encapsulation:** Each module (Streak, XP, Leaderboard) manages its own internal logic independently.
 - **Single Responsibility Principle:** Each observer class has one job and handles one specific reaction to the event.
-

Pattern 4 — Flyweight (Structural)

Pattern Name: Flyweight: reduces memory usage by sharing as much data as possible between similar objects. Instead of creating a new object every time, it reuses existing ones by separating intrinsic state (shared, stored once) from extrinsic state (context-specific, passed at runtime).

Category: Structural: these patterns deal with how classes are organized to make the overall system more efficient and easier to handle.

Where and How Does This Pattern Appear in Duolingo?

This pattern is used in Duolingo's Exercise Rendering System to manage the large number of visual elements displayed during lessons. During a lesson, Duolingo renders hundreds of letter tiles, word cards, characters, and icon elements on screen. Instead of creating a completely new object for every tile every time a lesson loads, it uses a Flyweight approach to reuse shared objects.

The intrinsic state, things like the letter's font, image asset, and audio reference is stored once inside a LetterTileFlyweight class and shared across all tiles of that letter. The extrinsic state like the tile's current position on screen or whether it is selected, is passed in at runtime and not stored inside the flyweight itself. A TilePool class acts as a cache, returning an already created flyweight when one exists instead of instantiating a new one.

Classes and Modules Involved

- **LetterTileFlyweight (flyweight class):** stores the shared intrinsic state (font, image, audio).
- **TilePool (flyweight factory):** manages and caches flyweight instances, returns existing ones when available.
- **ExerciseRenderer (client):** supplies extrinsic state like position and selection status at runtime.

Why Flyweight? What Problem Does It Solve?

Without the Flyweight pattern, the app would create a completely separate object for every single tile in every exercise, each one storing the same font, image, and audio data repeatedly. On low-end devices with limited RAM, this would cause serious performance issues or crashes. Flyweight solves this by making sure the heavy, repeated data is only stored once and shared

across all instances that need it, keeping memory usage low without changing how the app behaves from the outside.

OOP Principles Reinforced

- **Encapsulation** : The shared intrinsic state is neatly encapsulated inside the flyweight object. The client code does not need to know how the shared data is stored or managed.
- **Abstraction** : The ExerciseRenderer interacts with flyweight objects through a common interface without worrying about the internal details of how data is shared or cached behind the scenes.

Pattern 5 — Memento (Behavioral)

Pattern Name: Memento: captures and externalises an object's internal state at a specific point in time, stores it as a lightweight snapshot object (the Memento), and allows the system to restore that state later, all without violating encapsulation. The object whose state is saved is the Originator. The object managing the saved snapshots is the Caretaker, which never reads or modifies the Memento's contents.

Category: Behavioral: these patterns are concerned with how objects communicate and coordinate responsibilities at runtime. The Memento pattern specifically governs how an object's historical state is preserved, enabling recovery of past states without exposing internal structure.

Where and How Does This Pattern Appear in Duolingo?

This pattern appears in Duolingo's Lesson Session Management System. Every time a user is inside an active lesson, Duolingo maintains a running LessonSession object (the Originator) that continuously tracks the current question index, the user's answer history, hearts remaining, XP earned, and time elapsed. As the user progresses, the LessonSessionManager (the Caretaker) periodically calls saveState() on the LessonSession, producing an immutable LessonMemento snapshot of those values at that exact moment.

This enables two real recovery scenarios. First, if a user accidentally leaves the app mid-lesson, the Caretaker retrieves the most recent LessonMemento and calls restoreState(), resuming the session at the exact question and streak where the user left off. Second, on retry-enabled lesson modes, a checkpoint Memento is saved before a question is submitted — if the answer is wrong and a retry is triggered, the session rolls back to that checkpoint without corrupting the rest of the session data.

Classes and Modules Involved

- **LessonSession (Originator)**: the active session object. Exposes saveState() which packages its current state into a LessonMemento, and restoreState(memento) which unpacks a given Memento back into itself.
- **LessonMemento (Memento)**: an immutable snapshot holding questionIndex, heartCount, xpEarned, answerHistory, and elapsedTime. Exposes no setters and its contents are opaque to everything except the LessonSession that created it.

- **LessonSessionManager (Caretaker):** decides when to save and when to restore. Holds a stack of LessonMemento objects enabling multiple undo levels, but never inspects their contents.

Why Memento? What Problem Does It Solve?

Without the Memento pattern, the LessonSessionManager would need to directly access every private field of LessonSession to save and restore state, a serious violation of encapsulation. Any change to LessonSession's internal fields would then break the save/restore logic in LessonSessionManager. The Memento pattern solves this by letting the LessonSession itself control how its state is packaged and unpackaged. The Caretaker remains completely ignorant of the internal details, and adding new state fields only requires updating saveState() and restoreState() inside that one class, nothing else in the system changes.

OOP Principles Reinforced

- **Encapsulation:** The LessonMemento's internal fields are private and inaccessible to the Caretaker. Only the LessonSession has the authority to read from or write to a Memento.
- **Single Responsibility Principle:** LessonSession manages lesson logic, LessonMemento stores state, and LessonSessionManager coordinates saving and restoring. No class oversteps into another's domain.
- **Open/Closed Principle:** When new fields are added to LessonSession, only saveState() and restoreState() need updating inside that single class. The Caretaker and Memento interface require zero modification.

Pattern 6 — Abstract Factory (Creational)

Pattern Name: Abstract Factory: provides an interface for creating families of related objects that are guaranteed to work together, without specifying their concrete classes. Instead of creating one object at a time, an entire compatible family of objects is created through a single factory.

Category: Creational: these patterns deal with how objects are created in a controlled and organized way. Abstract Factory specifically handles the creation of entire families of related objects, ensuring everything produced together is always compatible.

Where and How Does This Pattern Appear in Duolingo?

This pattern appears in Duolingo's UI Rendering System. Duolingo runs on three platforms: Android, iOS, and Web and every screen contains UI components like buttons, text fields, progress bars, and dialog boxes. Each of these components looks and behaves differently depending on the platform. An Android Button follows Material Design guidelines with ripple animations, an iOS Button follows Apple's Human Interface Guidelines, and a Web Button includes hover effects and keyboard accessibility. These components must always belong together as a consistent family, mixing an Android Button with an iOS TextField would make the app look completely broken.

There is one abstract factory interface called UIComponentFactory that declares creation methods for every UI component the app needs createButton(), createTextField(),

createProgressBar(), and createDialogBox(). Each platform has its own concrete factory: AndroidUIFactory, iOSUIFactory, and WebUIFactory. When Duolingo launches on Android, the system instantiates AndroidUIFactory, which creates the entire family of Android-specific components, all guaranteed to be compatible. The ScreenRenderer receives the factory through the abstract interface and calls factory.createButton(), it never knows or cares whether it is an AndroidButton, iOSButton, or WebButton.

Classes and Modules Involved

- **UIComponentFactory (abstract base class):** declares all creation methods: createButton(), createTextField(), createProgressBar(), createDialogBox().
- **AndroidUIFactory, iOSUIFactory, WebUIFactory (concrete factories):** each implements UIComponentFactory and creates its own complete family of platform-specific components.
- **Button, TextField, ProgressBar (abstract products):** declare the interface for each component type.
- **AndroidButton, iOSButton, WebButton etc. (concrete products):** platform-specific implementations of each component.
- **ScreenRenderer (client):** holds a UIComponentFactory reference and calls creation methods on it, never knowing which platform it is running on.

Why Abstract Factory? What Problem Does It Solve?

Without this pattern, nothing stops the system from accidentally mixing components from different platforms, an AndroidButton paired with an iOSTextField is completely incompatible. On top of that, the ScreenRenderer would be filled with repeated platform checks for every single component across every screen. Abstract Factory solves both problems: the factory guarantees that all components always belong to the same compatible family, making incompatible combinations impossible. Adding a new platform like Smart TV only requires one new SmartTVUIFactory class, no existing code needs to be touched.

OOP Principles Reinforced

- **Abstraction:** ScreenRenderer only interacts through abstract interfaces. It never touches any concrete class like AndroidButton or iOSTextField.
 - **Encapsulation:** All creation logic for each platform is fully encapsulated inside its own factory class.
 - **Polymorphism:** All three factories implement the same UIComponentFactory interface. At runtime the correct factory executes automatically, same method call, different behavior, no if-else chains.
 - **Open/Closed Principle:** Adding a new platform only requires one new factory class implementing UIComponentFactory. No existing screens, renderers, or factories need any modification.
-

Pattern 7 — Strategy (Behavioral)

Pattern Name: Strategy: defines a family of algorithms, encapsulates each one in a separate class, and allows them to be interchangeable at runtime without changing the object that uses them. Same goal, different strategies.

Category: Behavioral: these patterns manage how objects communicate and share responsibility. Strategy specifically manages how an algorithm is selected and executed at runtime.

Where and How Does This Pattern Appear in Duolingo?

Strategy appears in Duolingo in two places.

Application 1 - Exercise Evaluation Strategy

Duolingo has many different exercise types and every one requires a completely different algorithm to evaluate the user's answer. A Multiple Choice Exercise simply checks if the selected option matches the correct answer. A Translation Exercise compares the user's typed text against acceptable answers, handles typos, punctuation differences, capitalization, and alternate valid translations. A Speaking Exercise captures audio, passes it through a speech recognition engine, converts it into phonemes, and compares pronunciation. A Matching Exercise checks whether all connected pairs are correct.

With Strategy, each algorithm is completely isolated in its own class. The Exercise object holds a reference to an EvaluationStrategy and calls evaluate(userAnswer) on it, never knowing which algorithm is running. Adding a new exercise type means one new class, zero changes anywhere else.

Application 2 - Adaptive Learning & Difficulty Strategy

Duolingo constantly monitors the user's skill level, streak, and performance and adjusts difficulty accordingly. The teaching logic changes based on where the user stands but the LessonManager stays exactly the same. BeginnerStrategy returns simple picture matching and heavily hinted exercises. StandardProgressionStrategy gradually introduces new vocabulary and slowly reduces hints. HardModeStrategy returns full sentence typing with zero hints and stricter answer checking. The strategy swaps instantly when the user's level changes, LessonManager code never changes at all.

Classes and Modules Involved

- **EvaluationStrategy:** abstract base class. Declares evaluate(userAnswer) that every concrete strategy must implement.
- **MultipleChoiceEvaluator:** checks if the selected option matches the correct answer.
- **TranslationEvaluator:** implements string matching with typo tolerance, punctuation handling, capitalization checks, and alternate answer verification.
- **SpeechEvaluator:** captures audio, passes through a speech recognition engine, converts to phonemes, and compares pronunciation.
- **MatchingEvaluator:** checks whether all connected pairs are correct.
- **Exercise:** context class. Holds an EvaluationStrategy reference and calls evaluate(userAnswer). Never implements evaluation logic itself.

- **DifficultyStrategy**: abstract base class. Declares getNextExercise(userProfile).
- **BeginnerStrategy, StandardProgressionStrategy, HardModeStrategy**: concrete difficulty strategies.
- **LessonManager**: context class. Holds the active DifficultyStrategy and calls getNextExercise(userProfile).

Why Strategy? What Problem Does It Solve?

Without Strategy, all evaluation logic sits inside one massive Exercise class and all difficulty logic inside one massive LessonManager. Every time Duolingo adds a new mode like Expert Mode, developers must go back into these classes and add more branches, risking breaking existing behavior. You also cannot test any single algorithm in isolation since everything is stuck inside one giant class. Swapping difficulty mid-session with rigid if-else blocks inside LessonManager is simply not possible.

```
// Inside Exercise:
if (exerciseType == "multipleChoice") { check selected option }
else if (exerciseType == "translation") { complex string matching... }
else if (exerciseType == "speaking")    { speech recognition... }
else if (exerciseType == "matching")    { check all pairs... }

// Inside LessonManager:
if (userLevel == "beginner") { simple pictures, word matching... }
else if (userLevel == "normal") { gradual vocabulary, reduce hints... }
else if (hardMode == true)    { full sentence typing, zero hints... }
```

With Strategy, each algorithm lives in its own isolated class. Adding Expert Mode means one new ExpertModeStrategy, nothing else changes. Swapping mid-session means plugging a different strategy into LessonManager at runtime, no restart, no broken code.

OOP Principles Reinforced

- **Encapsulation**: Each algorithm is fully encapsulated in its own class. Exercise and LessonManager just call the method, no idea what happens inside.
- **Polymorphism**: Exercise holds a reference of type EvaluationStrategy and calls evaluate(userAnswer) at runtime the correct version executes automatically.
- **Abstraction**: Exercise and LessonManager only interact with the abstract base classes, never with any concrete class.
- **Open/Closed Principle**: Adding a DrawingExercise only requires one new DrawingEvaluator class. Nothing else changes.

Pattern 8 — Factory Method (Creational)

Pattern Name: Factory Method: defines an interface for creating an object but allows subclasses to alter the type of objects that will be created. Instead of one central factory deciding with if/switch, the decision of which concrete class to instantiate is delegated to specialized creator subclasses by overriding a factory method.

Category: Creational: these patterns abstract the instantiation process, helping make a system independent of how its objects are created, composed, and represented.

Where and How Does This Pattern Appear in Duolingo?

This pattern appears in Duolingo's lesson/exercise delivery flow where different lesson modes generate different exercise experiences. For example, a "Listening-focused lesson" repeatedly generates `ListeningExercise` objects, a "Speaking-focused lesson" generates `SpeakingExercise` objects, and a "Mixed practice lesson" may generate a different exercise type depending on the lesson subtype. Instead of the `LessonController` deciding the class to instantiate, Duolingo can model each lesson mode as a `Creator` that overrides a factory method (e.g., `createExercise()`) to produce the correct `Exercise` subtype.

Classes and Modules Involved

- **Exercise (abstract base class):** defines the common contract (`load()`, `validate()`, `calculateScore()`).
- **MultipleChoiceExercise, ListeningExercise, SpeakingExercise, TranslationExercise:** concrete implementations.
- **LessonGenerator (Creator / abstract class):** declares the factory method `createExercise()` and provides the common "run exercise" workflow that uses the returned `Exercise` through the interface.
- **ListeningLessonGenerator, SpeakingLessonGenerator, TranslationLessonGenerator, MultipleChoiceLessonGenerator (Concrete Creators):** override `createExercise()` to return the appropriate `Exercise` subtype.
- **LessonController:** works with `LessonGenerator` through the base type and triggers the session flow, without knowing which concrete `Exercise` class gets instantiated inside each generator.

Why Factory Method? What Problem Does It Solve?

Without Factory Method, the `LessonController` (or a single `ExerciseFactory`) would contain a growing if-else/switch statement to decide which `Exercise` to create, and adding a new lesson mode/exercise type would require modifying that central decision code. Factory Method removes this by pushing object creation into creator subclasses: the base `Creator` (`LessonGenerator`) defines the stable workflow (`load` → `validate` → `score`), while each `Concrete Creator` decides which specific `Exercise` to instantiate by overriding `createExercise()`. This makes the system easier to extend because new lesson modes can be added by creating a new `Concrete Creator` class rather than editing existing controller logic.

OOP Principles Reinforced

- **Polymorphism:** The `LessonController` interacts with lessons through the `LessonGenerator` interface, and the lesson flow interacts with exercises through the `Exercise` interface. The same calls (`exercise.load()`, `exercise.validate()`,

exercise.calculateScore()) work for all exercise types, and the correct behavior is chosen at runtime.

- **Open/Closed Principle:** The core flow (LessonGenerator's run logic and LessonController's session logic) stays unchanged. To extend behavior, you add a new Concrete Creator (e.g., StoryLessonGenerator that creates StoryExercise) and/or a new Exercise subclass, without modifying the existing control flow code.

Task C: C++ Implementation

The following section provides three separate C++ implementations, one for each design pattern: Chain of Responsibility, Composite, and Observer. Each implementation is self-contained with its own main() function and inline comments explaining every class and method. The UML diagrams in Task D present a combined structural view illustrating how all three patterns operate within the Duolingo system.

Implementation 1 — Chain of Responsibility Pattern

This implementation models the Answer Validation Pipeline in Duolingo. Each validator (EmptyCheck, MinLengthCheck, TypoCheck, StrictCheck) is an independent handler linked in a chain. Each one either stops the chain with a decision or passes the answer along to the next handler.

```
#include <iostream>
#include <string>
using namespace std;

// =====
// 1. HANDLER (Abstract Base Class)
// =====
class Validator {
protected:
    Validator* next; // next handler in the chain

public:
    Validator() : next(NULL) {}

    void setNext(Validator* n) { next = n; }

    // Returns true if accepted, false if rejected
    virtual bool validate(string userAnswer, string correctAnswer) = 0;

    virtual ~Validator() {}
};

// =====
// 2. CONCRETE HANDLERS (Duolingo-like validators)
// =====

// A) Empty Check
class EmptyCheck : public Validator {
```

```

public:
    bool validate(string userAnswer, string correctAnswer) override {
        if (userAnswer == "") {
            cout << "[EmptyCheck] Rejected: Answer is empty.\n";
            return false; // stop chain
        }
        // pass to next
        if (next != NULL) return next->validate(userAnswer, correctAnswer);
        return false;
    }
};

// B) Minimum Length Check
class MinLengthCheck : public Validator {
private:
    int minLen;

public:
    MinLengthCheck(int m) : minLen(m) {}

    bool validate(string userAnswer, string correctAnswer) override {
        if ((int)userAnswer.size() < minLen) {
            cout << "[MinLengthCheck] Rejected: Too short.\n";
            return false; // stop chain
        }
        if (next != NULL) return next->validate(userAnswer, correctAnswer);
        return false;
    }
};

// C) Typo Tolerance (very simple: allow 1 character difference in length)
class TypoCheck : public Validator {
public:
    bool validate(string userAnswer, string correctAnswer) override {
        int diff = (int)userAnswer.size() - (int)correctAnswer.size();
        if (diff < 0) diff = -diff;

        if (userAnswer != correctAnswer && diff <= 1) {
            cout << "[TypoCheck] Accepted with warning: Minor typo (partial accept).\n";
            return true; // stop chain (accept here)
        }
        if (next != NULL) return next->validate(userAnswer, correctAnswer);
        return false;
    }
};

// D) Strict Correctness Check (final decision)
class StrictCheck : public Validator {
public:
    bool validate(string userAnswer, string correctAnswer) override {
        if (userAnswer == correctAnswer) {
            cout << "[StrictCheck] Accepted: Exact match!\n";
            return true;
        }
        cout << "[StrictCheck] Rejected: Incorrect answer.\n";
        return false; // final handler decides
    }
};

// =====
// 3. MAIN (Build the chain and test)
// =====
int main() {
    // Create validators
    EmptyCheck empty;

```

```

MinLengthCheck minLen(2);
TypoCheck typo;
StrictCheck strict;

// Build chain: Empty -> MinLength -> Typo -> Strict
empty.setNext(&minLen);
minLen.setNext(&typo);
typo.setNext(&strict);

cout << "=== Duolingo Answer Validation (Chain of Responsibility) ===\n\n";

// Test cases
cout << "Case 1: Empty answer\n";
empty.validate("", "hello");

cout << "\nCase 2: Too short\n";
empty.validate("h", "hello");

cout << "\nCase 3: Minor typo (length diff <= 1)\n";
empty.validate("hell", "hello");

cout << "\nCase 4: Correct answer\n";
empty.validate("hello", "hello");

cout << "\nCase 5: Incorrect answer\n";
empty.validate("world", "hello");

return 0;
}

```

Implementation 2 — Composite Pattern

This implementation models the Course Catalogue Hierarchy in Duolingo. The entire learning tree — Course, Section, Unit, Lesson, and Exercise — is built using the Composite pattern. Every node implements the same `LearningComponent` interface, allowing the client to call `start()`, `getTotalXP()`, and `isCompleted()` on any node uniformly, whether it is a single Exercise or an entire Course.

```

#include <iostream>
#include <vector>
#include <string>
using namespace std;

// =====
// 1. COMPONENT: LearningComponent (Abstract Base Class)
// =====
class LearningComponent {
public:
    virtual void start() = 0;
    virtual bool isCompleted() = 0;
    virtual int getTotalXP() = 0;
    virtual string getTitle() = 0;
    virtual ~LearningComponent() {}
};

// =====
// 2. LEAF: Exercise (The smallest unit)
// =====
class Exercise : public LearningComponent {

```

```

private:
    string title;
    int xpValue;
    bool completed;

public:
    Exercise(string t, int xp) : title(t), xpValue(xp), completed(false) {}

    void start() override {
        cout << "    [Exercise] Starting: " << title << "..." << endl;
        completed = true; // Simulating completion
    }

    bool isCompleted() override { return completed; }
    int getTotalXP() override { return xpValue; }
    string getTitle() override { return title; }
};

// =====
// 3. COMPOSITE: Base Container (Common logic for Course, Unit, etc.)
// =====
class LearningContainer : public LearningComponent {
protected:
    string title;
    vector<LearningComponent*> children;

public:
    LearningContainer(string t) : title(t) {}

    void add(LearningComponent* component) {
        children.push_back(component);
    }

    string getTitle() override { return title; }

    // Start runs all children recursively
    void start() override {
        cout << "\n>>> Entering " << title << "..." << endl;
        for (auto child : children) {
            child->start();
        }
    }

    // Returns true only if ALL children are completed
    bool isCompleted() override {
        for (auto child : children) {
            if (!child->isCompleted()) return false;
        }
        return true;
    }

    // Sums up XP from all children all the way down
    int getTotalXP() override {
        int total = 0;
        for (auto child : children) {
            total += child->getTotalXP();
        }
        return total;
    }

    ~LearningContainer() {
        for (auto child : children) delete child;
    }
}

```

```

};

// --- Specific Composite Types (Hierarchy Levels) ---

class Lesson : public LearningContainer {
public:
    Lesson(string t) : LearningContainer(t) {}
};

class Unit : public LearningContainer {
public:
    Unit(string t) : LearningContainer(t) {}
};

class Section : public LearningContainer {
public:
    Section(string t) : LearningContainer(t) {}
};

class Course : public LearningContainer {
public:
    Course(string t) : LearningContainer(t) {}
};

// =====
// 4. MAIN - Building the Duolingo Tree
// =====
int main() {
    // A. Create Exercises (Leaves)
    Exercise* ex1 = new Exercise("Translate 'Hello'", 10);
    Exercise* ex2 = new Exercise("Match 'Adios'", 5);
    Exercise* ex3 = new Exercise("Speaking: 'Buenos dias'", 15);

    // B. Group into a Lesson (Composite)
    Lesson* lesson1 = new Lesson("Lesson 1: Greetings");
    lesson1->add(ex1);
    lesson1->add(ex2);

    // C. Group Lesson into a Unit
    Unit* unit1 = new Unit("Unit 1: The Basics");
    unit1->add(lesson1);

    // Adding another direct exercise to Unit (just to show flexibility)
    unit1->add(ex3);

    // D. Group Unit into a Section
    Section* section1 = new Section("Section: Intro to Spanish");
    section1->add(unit1);

    // E. Root: The Course
    Course* myCourse = new Course("Spanish for English Speakers");
    myCourse->add(section1);

    // --- EXECUTION ---
    cout << "COURSE TITLE: " << myCourse->getTitle() << endl;

    // Client doesn't care if it's a course or an exercise
    myCourse->start();

    cout << "\n--- FINAL STATS ---" << endl;
    cout << "Total Course XP: " << myCourse->getTotalXP() << " XP" << endl;
}

```

```

    cout << "Course Status: " << (myCourse->isCompleted() ? "Completed! ?" : "In
Progress... ?") << endl;

    delete myCourse; // Deletes the entire tree
    return 0;
}

```

Implementation 3 — Observer Pattern

This implementation models the Gamification and Progress System in Duolingo. When a lesson is completed, the LessonController notifies all registered observers automatically. Each observer — StreakObserver, LeagueObserver, HealthObserver, GemObserver, and QuestObserver — reacts independently to the lesson completion event without any direct coupling between the systems.

```

#include <iostream>
#include <vector>
#include <string>

using namespace std;

// =====
// 1. THE OBSERVER INTERFACE
// =====
class LessonObserver {
public:
    virtual void onLessonCompleted(string username, int xpEarned, bool perfectLesson) =
0;
    virtual ~LessonObserver() {}
};

// =====
// 2. CONCRETE OBSERVERS (Real Duolingo Systems)
// =====

// System 1: Streak Management
class StreakObserver : public LessonObserver {
public:
    void onLessonCompleted(string username, int xpEarned, bool perfectLesson) override {
        cout << "[STREAK] Checking timestamp... 24h since last lesson? Yes!" << endl;
        cout << "    >> Flame icon is glowing! Your streak is now 45 days." << endl;
    }
};

// System 2: League & Leaderboard (XP)
class LeagueObserver : public LessonObserver {
public:
    void onLessonCompleted(string username, int xpEarned, bool perfectLesson) override {
        int bonus = perfectLesson ? 5 : 0; // Combo bonus
        cout << "[LEAGUE] Added " << xpEarned + bonus << " XP to your Bronze League
score." << endl;
        cout << "    >> You moved up to 4th place! (Perfect Lesson Bonus: +" << bonus << "
XP)" << endl;
    }
};

// System 3: Health & Hearts (If they finish, maybe refill?)
class HealthObserver : public LessonObserver {
public:
    void onLessonCompleted(string username, int xpEarned, bool perfectLesson) override {
        if (perfectLesson) {
            cout << "[HEALTH] Perfect lesson! You earned back 1 Heart <3" << endl;
        }
    }
};

```

```

        } else {
            cout << "[HEALTH] Lesson finished. Health remains stable." << endl;
        }
    }
};

// System 4: Inventory & Gems (Lingots)
class GemObserver : public LessonObserver {
public:
    void onLessonCompleted(string username, int xpEarned, bool perfectLesson) override {
        cout << "[GEMS] Lesson rewards: +12 Gems added to your account." << endl;
    }
};

// System 5: Daily Quests Tracking
class QuestObserver : public LessonObserver {
public:
    void onLessonCompleted(string username, int xpEarned, bool perfectLesson) override {
        cout << "[QUESTS] Progress: 'Finish 1 Lesson' [1/1] - COMPLETED!" << endl;
        cout << "    >> Progress: 'Earn 50 XP' [20/50] - In Progress..." << endl;
    }
};

// =====
// 3. THE SUBJECT (LessonController)
// =====
class LessonController {
private:
    vector<LessonObserver*> observers;

public:
    void subscribe(LessonObserver* obs) {
        observers.push_back(obs);
    }

    void unsubscribe(LessonObserver* obs) {
        for (auto it = observers.begin(); it != observers.end(); ++it) {
            if (*it == obs) {
                observers.erase(it);
                break;
            }
        }
    }

    // Trigger the event
    void finishLesson(string username, int xp, bool perfect) {
        cout << "\n===== " << endl;
        cout << "    LESSON SUMMARY FOR: " << username << endl;
        cout << "===== " << endl;

        for (auto obs : observers) {
            obs->onLessonCompleted(username, xp, perfect);
        }
    }
};

// =====
// 4. MAIN - Running the Duo engine
// =====
int main() {
    LessonController duoEngine;

    // Create real systems
    StreakObserver streak;
    LeagueObserver league;
    HealthObserver health;
    GemObserver gems;

```

```
QuestObserver quests;

// Registering the systems
duoEngine.subscribe(&streak);
duoEngine.subscribe(&league);
duoEngine.subscribe(&health);
duoEngine.subscribe(&gems);
duoEngine.subscribe(&quests);

// Scenario A: Humaira finishes a lesson perfectly
duoEngine.finishLesson("Humaira", 15, true);

// Scenario B: User disables Gem notifications (Inventory updates)
cout << "\n[System] User muted shop notifications..." << endl;
duoEngine.unsubscribe(&gems);

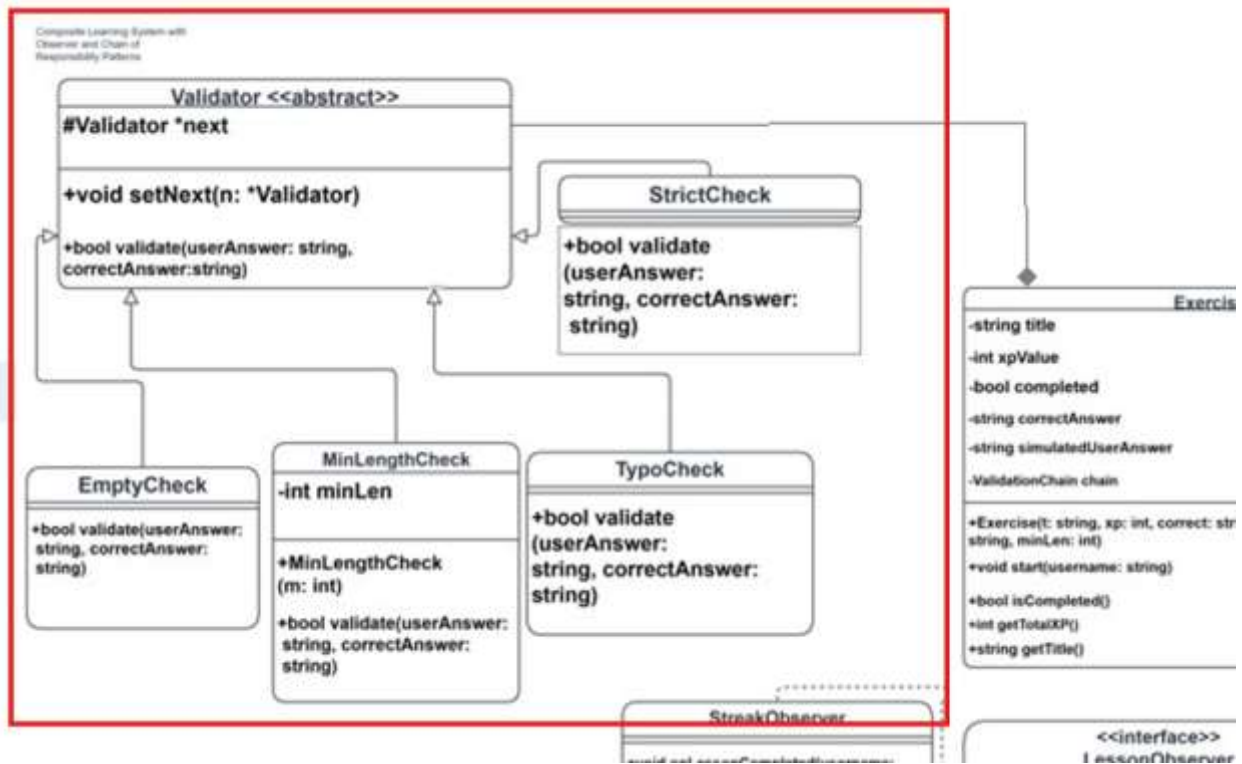
// Scenario C: Another lesson finished (Gems system won't fire)
duoEngine.finishLesson("Humaira", 10, false);

return 0;
}
```

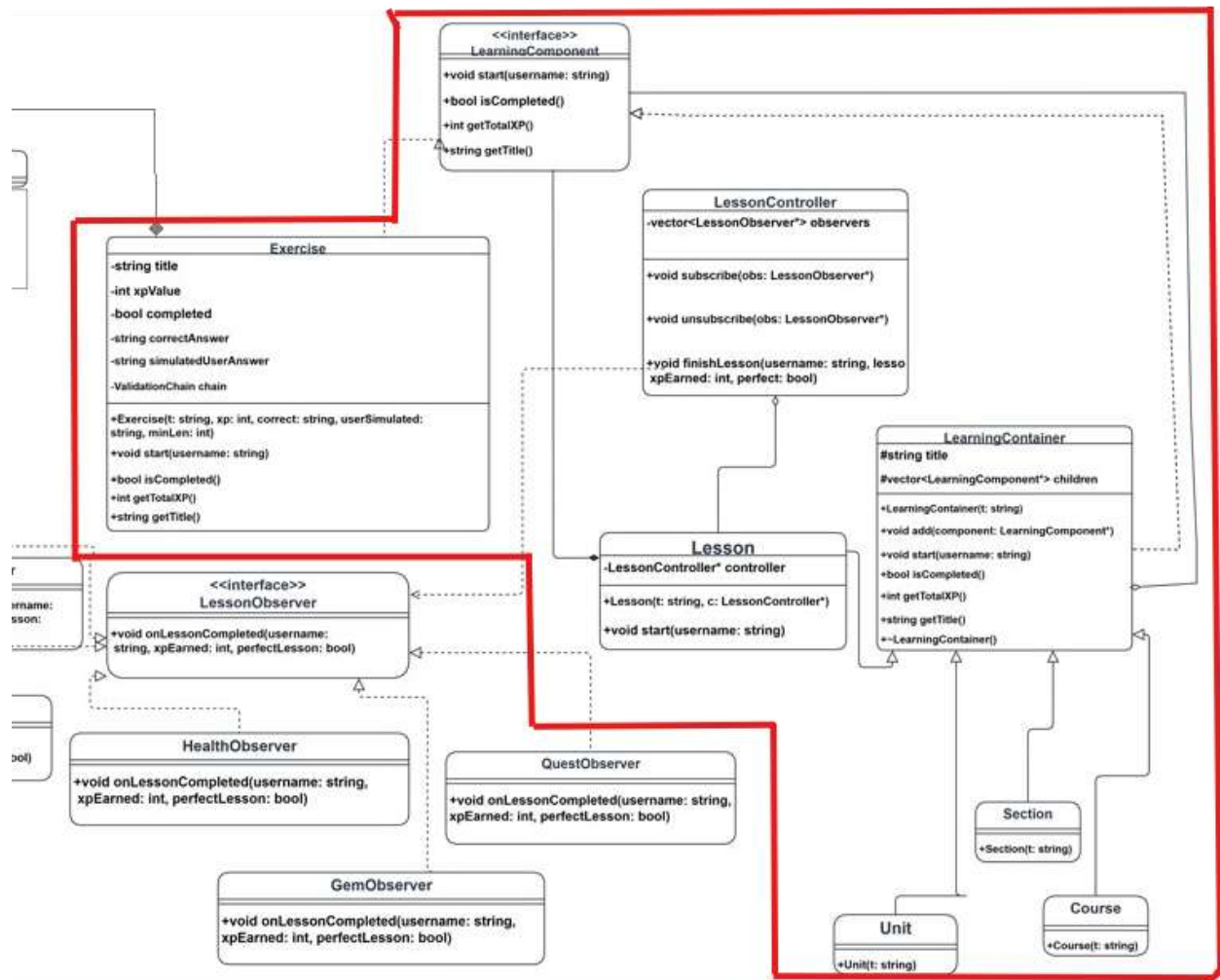
The UML class diagrams below present a combined structural view of all three implemented patterns: Chain of Responsibility, Composite, and Observer. Although each pattern operates in its own subsystem, the diagrams are presented together to show how the classes and interfaces across all three patterns relate to one another in the broader Duolingo architecture. Each diagram includes class names, key attributes, methods, access modifiers, and relationships using correct UML notation.

```
classDiagram
    class Validator {
        <<abstract>>
        #Validator *next
        +void setNext(n: *Validator)
        +bool validate(userAnswer: string, correctAnswer: string)
    }
    class EmptyCheck {
        +bool validate(userAnswer: string, correctAnswer: string)
    }
    class MinLengthCheck {
        -int minLen
        +bool validate(userAnswer: string, correctAnswer: string)
    }
    class TypoCheck {
        +bool validate(userAnswer: string, correctAnswer: string)
    }
    class StrictCheck {
        +bool validate(userAnswer: string, correctAnswer: string)
    }
    class Exercise {
        -string title
        -int xpValue
        -bool completed
        -string correctAnswer
        -string simulatedUserAnswer
        -ValidatorChain chain
        +Exercise() string, xp: int, correct: string, userSimulated: string, minLen: int
        +void start(username: string)
        +bool isCompleted()
        +int getTotalXP()
        +string getTitle()
    }
    class LessonObserver {
        <<interface>>
        +void onLessonCompleted(username: string, xpEarned: int, perfectLesson: bool)
    }
    class StreakObserver {
        +void onLessonCompleted(username: string, xpEarned: int, perfectLesson: bool)
    }
    class LeagueObserver {
        +void onLessonCompleted(username: string, xpEarned: int, perfectLesson: bool)
    }
    class HealthObserver {
        +void onLessonCompleted(username: string, xpEarned: int, perfectLesson: bool)
    }
    class GemObserver {
        +void onLessonCompleted(username: string, xpEarned: int, perfectLesson: bool)
    }
    class QuestObserver {
        +void onLessonCompleted(username: string, xpEarned: int, perfectLesson: bool)
    }
    class LearningComponent {
        <<interface>>
        +void start(username: string)
        +bool isCompleted()
        +int getTotalXP()
        +string getTitle()
    }
    class LessonController {
        -vector<LessonObserver> observers
        +void subscribe(obs: LessonObserver)
        +void unsubscribe(obs: LessonObserver)
        +void finish(Lesson username: string, lesson xpEarned: int, perfect: bool)
    }
    class Lesson {
        -LessonController* controller
        -Lesson() string, c: LessonController
        +void start(username: string)
    }
    class LearningContainer {
        #string title
        #vector<LearningComponent> children
        +LearningContainer() string
        +void add(component: LearningComponent)
        +void start(username: string)
        +bool isCompleted()
        +int getTotalXP()
        +string getTitle()
        +LearningComponent
    }
    class Section {
        -Section: string
    }
    class Unit {
        -Unit() string
    }
    class Course {
        -Course() string
    }
    Validator <|-- EmptyCheck
    Validator <|-- MinLengthCheck
    Validator <|-- TypoCheck
    Validator <|-- StrictCheck
    Validator o--> Exercise
    LessonObserver <|-- StreakObserver
    LessonObserver <|-- LeagueObserver
    LessonObserver <|-- HealthObserver
    LessonObserver <|-- GemObserver
    LessonObserver <|-- QuestObserver
    LearningComponent <|-- Lesson
    LearningComponent <|-- Section
    LearningComponent <|-- Unit
    LearningComponent <|-- Course
    LessonController o--> Lesson
    LessonController o--> LearningContainer
    Lesson o--> LessonController
    Lesson o--> LearningContainer
    LearningContainer o--> Section
    LearningContainer o--> Unit
    LearningContainer o--> Course
```

CHAIN OF RESPONSIBILITY UML DIAGRAM:



COMPOSITE UML DIAGRAM:



OBSERVER UML DIAGRAM:

